

17-313: Foundations of Software Engineering

Midterm Review – Mock Exam

Total: 70 points

Question 1: Software Engineering with AI

[8 points]

A software engineering team uses an AI coding agent to implement a new feature in an existing software project. The agent is given a goal and can search the repository, edit files, run tests, and create a pull request.

- (a) [3 points] Explain one advantage of using an AI coding agent instead of a traditional programming assistant such as code autocomplete. **An agent can autonomously explore the repository, modify code, run tools/tests, and iterate toward a goal, whereas a programming assistant primarily assists with individual coding tasks.**
- (b) [2 points] The team wants the AI agent to consistently follow the project's architecture, coding conventions, and testing requirements across different tasks. What mechanism discussed in class could the team use to provide this persistent project context? **AGENTS.md**
- (c) [3 points] The team also wants the agent to have access to external tools without giving it unrestricted access to the underlying system. What technology discussed in class is designed for this purpose? **Model Context Protocol (MCP)**

Question 2: Metrics and Measurements

[10 points]

The company. You work for BusMagic, a small, fictional technology company that specializes in creating custom transit apps for cities across the world.

These apps allow individuals to learn about and use local transit options by:

- (a) searching a map for directions for transit routes between destinations; and
- (b) selecting stops on the map and querying them for information about upcoming buses or trains.

The app accomplishes this by interfacing with and querying information from a city or country's existing transit infrastructure. Additionally, for cities whose transit systems support it, the app allows customers to pay fares using their phone.

The team. You are on one of several small teams within the company focusing on backend concerns; others are in charge of testing/QA and frontend UI. You work with two senior members with considerable mobile development experience, as well as three additional, more junior employees and a data scientist.

- (a) [4 points] BusMagic has a license for a tool that indexes the entire repository and has a plugin for your IDE. The tool creates a relative ranking to judge the number of lines of code written by each developer. This allows you to use the quantity of code written by each developer over time as a usable metric.

Your CTO suggests you use this tool to identify developers writing the least amount of code and then provide them mentoring and extra training.

What is the main problem with using “lines of code written” as the metric for identifying developers who need mentoring or extra training?

Lines of code is a poor metric because writing more code does not necessarily mean a developer is more productive or effective, and writing less code does not necessarily indicate that they need mentoring or training. Developers may work on tasks that naturally require different amounts of code, and good code may actually reduce the amount of code needed. Therefore, this metric does not reliably answer whether a developer needs additional support. The goal is to identify developers who need mentoring or extra training. A relevant question would be something like: “Which developers are having difficulty completing their assigned work effectively?” The proposed metric does not adequately answer this question because LOC measures code quantity, not developer effectiveness or difficulty. Therefore, the metric has a weak connection to the goal and is not an appropriate GQM metric for this purpose.

- (b) [6 points] For each of the following quality attributes, give a good metric that you can use to determine if the implemented features of the application (searching for directions and querying stops) are satisfying the associated quality requirement.

- i. Performance
- ii. Reliability
- iii. Usability
- iv. Accuracy
- v. Availability

Performance: Average response time for a directions search or stop query

Reliability: Percentage of searches/queries that complete successfully without an error

Usability: Average time taken by users to successfully find directions or information about a stop

Accuracy: Percentage of directions/stop information returned that matches the correct expected result

Availability: Percentage of time the directions and stop-query features are operational

Question 3: Case Study – Boeing 737 MAX [7 points]

- (a) [4 points] We have discussed a major case study involving Boeing, where process and culture concerns contributed to software failures.

Explain how process and organizational culture issues contributed to the Boeing 737 MAX failures. Identify at least two relevant factors.

The Boeing 737 MAX case showed how process and organizational culture issues can contribute to software failures. For example, schedule and competitive pressures contributed to decisions that prioritized minimizing delays and changes to the aircraft. There were also communication and training shortcomings, including inadequate communication about the MCAS system and insufficient pilot training. These factors contributed to the software being

developed, reviewed, and deployed without fully accounting for important failure scenarios and interactions between the system and its sensors.

- (b) [3 points] What is one process intervention you could implement in your company to reduce the likelihood of a similar type of failure? Explain how the intervention would help.

Implement a mandatory independent safety and software review before deployment, where engineers who were not directly involved in developing the feature review its requirements, assumptions, failure modes, and testing results. This can help identify safety-critical issues and ensure that concerns are addressed before the software is released.

Question 4: Process and Planning

[8 points]

You work for an on-demand beverage delivery company called BobaFetch. Customers in the Pittsburgh area download the BobaFetch mobile app and use it to order bubble tea and other drinks from local cafes. They pay for their drinks on the app at order time. Once the beverages are prepared, they are home delivered to customers by droids, which are small autonomous delivery robots. BobaFetch owns a fleet of droids that can be dispatched to cafes as orders are received and then routed to customers' homes.

The team is implementing and testing API call and usage tracking for the DaaS API. The subsystem should record each API call, the associated customer, timestamp, and droid/job identifier, so that the information can later be used to calculate charges based on both the number of API calls and the time the droid is used.

You find that no one is evaluating risks for BobaFetch. Following the process you learned in your undergraduate Software Engineering course, you decide to follow the process of identifying risks, assessing them, and developing a mitigation strategy.

1. [4 points] Identify one risk associated with the API usage-tracking subsystem, assess its likelihood, and provide a mitigation strategy.

Incorrect billing due to the system failing to record API calls or droid usage accurately. To mitigate, Add comprehensive usage logging, validate billing data, and test billing under different failure scenarios.

2. [4 points] Identify a second risk associated with the API usage-tracking subsystem, assess its likelihood, and provide a mitigation strategy.

A large number of API requests could overload the DaaS API and delay droid dispatches. To mitigate, implement rate limiting, monitoring, and load testing to ensure the API can handle expected traffic.

Question 5: Teams and Communication

[7 points]

In the BobaFetch onboarding document, you want to provide an example of a poorly composed issue that can highlight the need for well-composed GitHub issues.

Consider the following issue:

The droid isn't working properly.

It was working before, but now there is a problem when it tries to make deliveries. Please fix it as soon as possible. I think something changed in the API. Maybe the routing code is broken. Let me know if you need anything else.

- (a) [4 points] Identify at least four ways in which this issue could be improved. For each improvement, explain what information should be added or changed.

Context: The issue does not explain the conditions that led to the issue. It only says that the droid “isn't working properly.” It should explain when the problem started, what the droid was doing, and under what circumstances the problem occurs.

Problem or idea: The actual problem is too vague. “There is a problem when it tries to make deliveries” does not tell developers what is going wrong. It should describe the specific unexpected behavior, such as “the droid stops after leaving the café and does not continue to the customer's destination.”

Previous attempts to solve: The issue does not explain what has already been tried. Developers might repeat unsuccessful debugging steps. It should document previous attempts, such as restarting the droid, checking the routing service, or reproducing the problem with another delivery.

Solution or next step: The issue simply says “Please fix it” and speculates that the API or routing code might be broken. It should provide a concrete next step, such as investigating the routing API logs or reproducing the failure using a specific delivery request.

- (b) [3 points] Describe ways in which this issue can be improved

Add context: Explain when the problem started and under what circumstances it occurs.

Clearly describe the problem: Replace vague statements like “the droid isn't working properly” with the specific observed behavior.

Document previous attempts: Explain what has already been tried to diagnose or fix the problem.

Provide a concrete next step or solution: Suggest what should be investigated or changed instead of simply saying “fix it.”

Include technical details: Provide relevant environment settings, software/API versions, error messages, logs, screenshots, or code examples.

Question 6: Design Documents

[8 points]

Scenario

A team is developing an online shopping application. The team proposes adding a **Personalized Promotions** feature that recommends discounts to users based on their previous purchases.

The team creates the following design document excerpt:

Personalized Promotions

We will use AI to recommend personalized discounts to customers. The system will analyze customer information and show each customer the promotions that are most relevant to them. The recommendations should be fast and secure. We will implement the feature using a recommendation model and store the necessary customer data. The team will decide the exact implementation details during development.

- (a) [2 points] Identify two ambiguities in the proposed design. For each ambiguity, explain how two developers could interpret the design differently.

1. “Customer information” is ambiguous. One developer might interpret this as purchase history only, while another might include names, locations, browsing history, or other personal information.

2. “Relevant promotions” is ambiguous. One developer might recommend promotions based on previous purchases, while another might use browsing behavior or demographic information.

- (b) [3 points] Identify each of the following and explain why it belongs in the design document:

- i. One functional requirement that should be specified.
- ii. One quality requirement that should be specified.
- iii. One privacy or security concern that should be addressed.

i. Functional requirement: The system should generate and display personalized promotions based on a user’s purchase history. This specifies what the system must do.

ii. Quality requirement: The system should generate recommendations within a specified amount of time, such as 500 ms. This makes the vague requirement that recommendations be “fast” measurable.

iii. Privacy concern: The design should specify what customer data is collected, how it is stored, and who can access it. This is important because the feature uses customer information.

- (c) [3 points] The team is considering two approaches:

- **Approach A:** Store detailed purchase histories and use them to generate personalized promotions.
- **Approach B:** Generate promotions using only a user’s recent purchases and less detailed customer information.

Describe one advantage and one disadvantage of each approach, and explain what tradeoff the team is making.

Approach A: An advantage is that detailed purchase histories could allow the system to generate more personalized promotions. A disadvantage is that storing more customer information creates greater privacy and storage concerns.

Approach B: An advantage is that it uses less customer information, potentially reducing privacy concerns. A disadvantage is that the recommendations may be less personalized because the system has less information about the user.

Question 7: Software Specifications

[10 points]

The online shopping application development team asks an AI system to provide specifications for the recommendation feature they are building. Below are the AI-generated specifications:

1. The system should provide users with personalized promotions based on their purchase history.
2. Promotions should be relevant to each user.
3. The system should respond quickly when displaying promotions.
4. Customer data should be stored securely.
5. The system should use AI to determine the best promotions for each customer.
6. Promotions should be updated regularly based on the customer's behavior.
7. The system should show the most appropriate promotion to the user.
8. The system should work for all customers and provide a good user experience.

Identify **five problems** with the AI-generated specifications. For each problem:

1. Identify what decision, behavior, or requirement is left unclear.
 2. Explain why this is a problem for a software specification.
 3. Rewrite the corresponding specification so that it is more precise and testable.
1. "Personalized" / "relevant" is ambiguous. The specification does not define what information determines whether a promotion is relevant. Different developers or AI agents could make different assumptions.
 2. "Respond quickly" is not precise. It does not specify an acceptable response time. One developer might interpret "quickly" as 100 ms while another might interpret it as 2 seconds.
 3. "Stored securely" is underspecified. It does not state what security constraints must be satisfied, such as who may access the data or what data may be stored.

4. “Updated regularly” is ambiguous. It does not specify how often promotions should be updated or what event should trigger an update.
5. “Most appropriate promotion” is ambiguous. The specification does not define how promotions are selected when multiple promotions could be appropriate, or what happens when no promotion is appropriate.

1. Original: “The system should provide users with personalized promotions based on their purchase history.”

Improved: “The system shall recommend promotions based on products the user has purchased within the previous 12 months.”

2. Original: “Promotions should be relevant to each user.”

Improved: “A promotion shall be considered relevant if it applies to a product category in which the user has made at least one purchase within the previous 12 months.”

3. Original: “The system should respond quickly when displaying promotions.”

Improved: “The system shall return the user’s recommended promotions within 500 ms for at least 99% of requests.”

4. Original: “Promotions should be updated regularly based on the customer’s behavior.”

Improved: “The system shall recalculate a user’s recommended promotions whenever the user completes a purchase.”

5. Original: “The system should show the most appropriate promotion to the user.”

Improved: “The system shall display up to three eligible promotions, ordered by the promotion’s calculated relevance score. If no eligible promotion exists, the system shall display no personalized promotion.”

Question 8: Software Quality

[7 points]

The BobaFetch team has implemented the API usage-tracking subsystem. The system records the customer, timestamp, API call, and droid/job identifier for every DaaS API request.

The team runs 500 automated tests, and all of them pass. One developer concludes:

“Since all 500 tests passed, we can be confident that the usage-tracking subsystem has no defects.”

- (a) [2 points] Is the developer’s conclusion justified? Explain why or why not. **No. Testing can demonstrate the presence of defects, but passing tests cannot demonstrate their absence. The 500 tests only show that the tested cases behaved as expected; other inputs or interactions may still expose defects.**
- (b) [2 points] The team wants to improve its test suite. They decide to test inputs such as missing customer IDs, empty job identifiers, timestamps at unusual boundaries, and extremely large numbers of API calls. What test design technique could help them systematically select useful test cases from these input categories? Explain how the technique would be applied. **Equivalence partitioning. The team can divide inputs into classes expected to have similar**

behavior, such as valid IDs, missing IDs, empty IDs, and unusually large values, and select representative cases from each class.

- (c) **[3 points]** The team discovers that the system fails only when a customer has multiple active droids **and** those droids make API calls at the same time. What testing technique could help identify this type of defect? Explain why this technique is appropriate. **Pairwise testing.** The defect depends on the interaction between two conditions: multiple active droids and simultaneous API calls. Pairwise testing systematically tests combinations of parameter values without requiring the full cross-product of all possible combinations.

Question 9: Fuzz Testing

[5 points]

The team wants to use fuzz testing to test the **Personalized Promotions** feature.

How would you apply fuzz testing to this feature? Describe the types of inputs you would generate and what kinds of behavior you would observe or verify.

Generate many random, unexpected, or invalid purchase histories as inputs to the Personalized Promotions feature and observe whether the application handles them correctly. For example, generate users with empty purchase histories, very large purchase histories, missing product information, negative quantities/prices, or unusual product data. Check whether the system crashes, produces invalid promotions, or otherwise behaves unexpectedly.